

UCS503:

CarbonRoute

Uncertainty-Aware Carbon-Aware Batch Scheduling

Team Name: TriFlux

Yuvika Nagpal • 1024030141

Kumkum Gupta • 1024030144

Aaneya Sabharwal • 1024030147

Batch: 3C15

TABLE OF CONTENTS

01 Introduction

02 Problem Statement & Research Gap

03 Project Scope & Objectives

04 Proposed Functions

05 Target Users

06 System Features

07 Interfaces & Technical Architecture

08 Performance Goals

09 Security Measures

10 Quality Attributes

11 Scheduling Policies & Baselines

12 Job Classes & Service Levels

13 Uncertainty-Aware Scheduling Method

14 Deadline-Risk Estimation & Calibration

15 Optimization & Risk-Aware Formulation

16 Data Sources & Trace Construction

17 Experiment Design & Scale

18 Evaluation & Reproducibility

19 Real Workload Connector & Public Website

20 Initial Planning

21 Team Responsibilities & Technical Risks

22 Conclusion

23 References

Introduction

Project overview: CarbonRoute is a software-only system for scheduling flexible batch workloads while reducing realized carbon emissions under uncertain carbon-intensity forecasts.

Why it matters: Carbon intensity varies across time and regions. Flexible workloads can be shifted, but forecast errors can make apparently clean schedules risky.

Presentation objective: Define the project scope, scheduling method, uncertainty/risk framework, system architecture, evaluation plan, deployment path and initial milestones.

Problem Statement & Research Gap

Problem: Existing carbon-aware scheduling relies on forecasts that may contain bias, random error or missing values. Capacity limits, price changes, workload surges and tight deadlines can further affect decisions.

Research gap: CarbonRoute focuses on forecast-versus-realized analysis, deadline-violation probability, risk calibration, controlled stress testing and reproducible comparison of scheduling policies.

Positioning: CarbonRoute does not claim to invent carbon-aware scheduling; its contribution is an uncertainty-aware, calibrated and reproducible scheduling/evaluation framework.

Project Scope & Objectives

In scope: Multi-region simulation, carbon traces, forecast-error modelling, cost/capacity modelling, scheduling algorithms, deadline-risk estimation, calibration, benchmark, dashboard, API, workload connector and public website.

Core objectives: Implement required baselines; develop uncertainty-aware scheduling; estimate and calibrate deadline risk; run controlled experiments; report statistical comparisons; connect the API to a real software workload interface.

Out of scope: Hardware/sensors, physical data-centre control, production workload migration, complete cloud-provider replacement, LLM-based scheduling and production-scale orchestration.

Project constraint: The project is completely software-based. No hardware component or LLM is required for scheduling logic.

Proposed Functions

1. **Job submission:** Accept workload requirements such as release time, duration, resources, deadline, cost limit, region and risk tolerance.
2. **Scheduling:** Generate candidate execution windows and select a feasible low-carbon option.
3. **Risk estimation:** Estimate the probability of deadline violation using forecast uncertainty and workload conditions.
4. **Evaluation:** Record realized outcomes and compare policies using carbon, deadline, risk and statistical metrics.
5. **Dispatch:** After the framework is stable, send a containerized batch workload to a real software connector.

Target Users

Cloud engineers: Schedule flexible workloads while controlling carbon and cost.

Data/ML engineers: Run training and analytics jobs within deadlines.

DevOps teams: Use an API to make scheduling decisions for containerized jobs.

Sustainability teams: Measure actual carbon reduction using quantitative evidence.

Researchers and students: Reproduce and compare scheduling policies and risk calibration.

System Features

Carbon-aware scheduling: Select lower-carbon feasible execution opportunities.

Uncertainty modelling: Represent forecast error rather than treating predictions as certain.

Deadline-risk estimation: Predict the probability of missing a deadline.

Risk calibration: Compare predicted risk with observed violation frequency using Brier Score, calibration error and reliability plots.

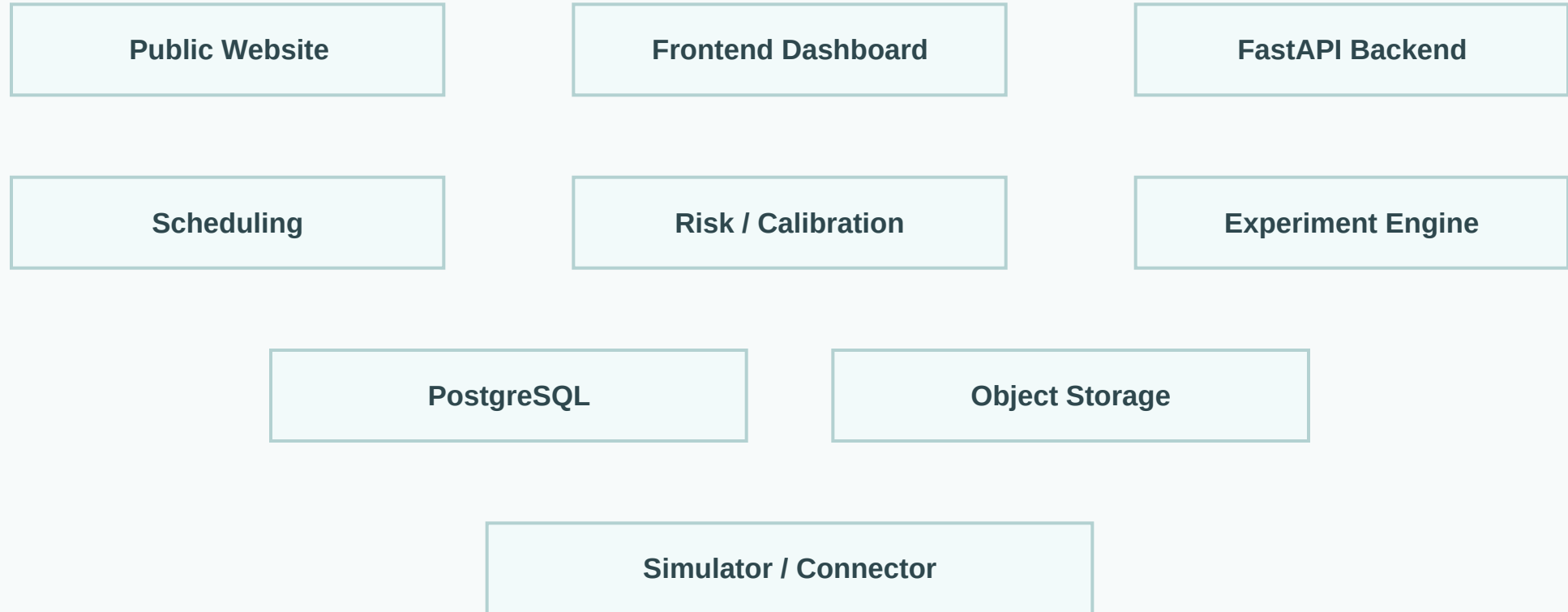
Benchmarking: Compare immediate, EDF, cost-aware, CarbonAware, CarbonRoute and oracle policies.

Reproducibility: Version traces, workloads, seeds, algorithms, parameters and solver limits.

Dashboard & API: Configure experiments, inspect decisions and publish results.

Workload connector: Dispatch a trusted containerized batch job.

Interfaces & Technical Architecture



Performance Goals

Scheduling response: Target low API latency for interactive scheduling decisions.

Experiment throughput: Support repeated runs across regions, workloads, forecast-error levels and seeds.

Data handling: Store traces and results efficiently while preserving version metadata.

Reliability: Detect infeasible schedules and connector failures without silently producing invalid results.

Achievement strategy: Modular Python services, efficient data processing, indexed result storage, bounded solver execution and continuous testing.

Security Measures

Authentication: Secure login for the administrative interface.

Authorization: Role-based access for publishing and project administration.

Data protection: Hashed passwords, secure object-storage access and protected credentials.

File security: Validate file types and sizes before storage.

Backend security: Validate API inputs, enforce authorization and maintain audit logs.

Workload security: Use trusted container images only; do not execute arbitrary unsafe code.

Quality Attributes

Reliability: Constraint validation, failure handling, unit/integration tests and reproducibility checks.

Usability: Clear scheduling explanations, visible risk values and an intuitive experiment dashboard.

Maintainability: Modular scheduler/risk/API components, version-controlled data and documented workflows.

Consistency: Run the same experiment configuration and seed to reproduce the same main results.

Testing: Scheduling correctness, API, constraint, performance and experiment-consistency tests.

Scheduling Policies & Baselines

Immediate execution: Runs the job as soon as resources are available.

Earliest Deadline First (EDF): Prioritizes jobs with the closest deadlines.

Cost-aware scheduling: Selects the cheapest feasible execution option.

CarbonAware Scheduler: Required external baseline for carbon-aware scheduling.

CarbonRoute: Proposed uncertainty-aware scheduler using carbon, uncertainty, risk and constraints.

Realized-data oracle: Uses future realized carbon only as a best-possible reference; it is not deployable.

Job Classes & Service Levels

Strict-deadline jobs: Deadlines cannot be violated. If no feasible schedule exists, the system reports infeasibility and explains the conflicting constraints. Example: A backup must finish before 11:00 PM.

Probabilistic-service jobs: A deadline may be treated as a service-level requirement. Example: A data-analysis job should finish before 10:00 PM with at least 95% probability.

Risk constraint: $P(\text{deadline violation}) \leq 0.05$ for a 95% service requirement.

Decision output: The estimated deadline risk is displayed alongside the selected schedule.

Uncertainty-Aware Scheduling Method

Method: For every candidate window, combine carbon forecast + forecast uncertainty + workload conditions to estimate deadline risk, apply hard/risk constraints and select a feasible schedule.

Feasibility demonstration: Use the same job and the same predicted carbon values while changing only forecast uncertainty.

Low uncertainty: Example risk 2% -> delay to cleaner period.

High uncertainty: Example risk 18% -> start earlier.

Validation: Repeat the scenarios using realized outcomes and compare predicted risk with observed deadline violations.

Deadline-Risk Estimation & Calibration

Risk being estimated: Probability that a scheduled job will violate its deadline.

Calibration example: If 100 comparable jobs are predicted at 20% risk, about 20 should violate the deadline for a well-calibrated model.

Brier Score: Measures probabilistic prediction error.

Calibration Error / ECE: Measures the difference between predicted probability and observed outcome frequency.

Reliability plot: Predicted deadline-violation probability vs actual violation frequency; good calibration is close to the diagonal.

Required outputs: Brier Score + calibration error + reliability plot.

Optimization & Risk-Aware Formulation

Objective factors: Expected/realized carbon, cloud cost, delay, deadline risk and forecast uncertainty.

Important design choice: CarbonRoute will not rely only on a weighted objective.

Constrained formulation: For probabilistic-service jobs, enforce $P(\text{deadline violation}) \leq \epsilon$.

Example: For a 95% service requirement, $P(\text{deadline violation}) \leq 0.05$.

Sensitivity analysis: Vary risk thresholds, objective weights and forecast-error levels to test how decisions and outcomes change.

Data Sources & Trace Construction

Data source to freeze: The current project plan references Electricity Maps. Before implementation, the team will freeze the provider, licence, supported regions, API/data limits and exact data fields.

Required specification: Region, time resolution, forecast horizon, missing-data treatment and forecast-realized alignment.

Trace fields: Timestamp, region, forecast carbon, realized carbon, energy cost, available CPU/memory and capacity status.

Versioning: Every trace receives a version identifier so experiments remain reproducible.

Alignment principle: Forecast values must be paired with the realized value for the same target time and region.

Experiment Design & Scale

Initial scale: 3 regions; 30-minute resolution; 24-hour forecast horizon; 3 workload classes.

Workload volume: 100, 500 and 1,000 jobs per trace.

Forecast-error levels: 0%, 5%, 10%, 20% and 30%.

Repeated seeds: At least 30 seeds per configuration.

Workload classes: Flexible batch, tight-deadline and mixed workloads.

Stress scenarios: Forecast bias, random error, missing forecasts, capacity loss, price changes, workload surges and tight deadlines.

Evaluation & Reproducibility

Carbon: Total realized emissions and carbon reduction percentage.

Deadline: Deadline-violation rate and mean job delay.

Risk: Brier Score, calibration error and reliability plots.

Comparison: Regret against oracle, paired algorithm comparisons and confidence intervals.

System: Scheduler runtime, resource utilization and completed jobs.

Reproducibility record: Trace version, workload version, seed, algorithm version, parameters and solver limit.

Workflow: The repository will provide one documented workflow to regenerate the main benchmark results, plots and summary tables.

Real Workload Connector & Public Website

Real connector: After the simulator and scheduling framework are stable, connect to at least one real software workload interface.

Planned connector: Kubernetes Jobs (preferred), using a small trusted containerized batch workload.

Final flow: Containerized job -> CarbonRoute API -> scheduling decision -> constraint/risk explanation -> Kubernetes dispatch -> execution status.

Public website: Homepage, Planning Presentation V1, future presentation/demo pages, admin interface and permanent versioned presentation pages.

Website requirement: Backend must actually handle authentication, upload, metadata, publishing and preservation of previous versions.

Initial Planning — Feasibility, Gantt & Milestones

Weeks	Phase	Milestone
1–2	Requirements & research	Scope frozen
3–4	Job/region/trace models	Simulation base
5–6	Immediate + EDF + cost	Baselines
7–8	CarbonAware + Oracle	Baseline set complete
9–10	Forecast error + versioning	Trace pipeline
11–12	Uncertainty + calibration	Risk model
13	Stress testing	Robustness results
14	API + dashboard	Hosted prototype
15	Repeated experiments	Statistical results
16	Testing + deployment + connector	End-to-end demo
17	Final results + viva	Final delivery

Team Responsibilities & Technical Risks

Yuvika — Simulation & Traces: Simulation architecture, workload generation, carbon/workload traces and forecast-error scenarios.

Kumkum — Scheduling & Calibration: Scheduling algorithms, uncertainty model, deadline-risk estimation, calibration and baseline comparison.

Aaneya — API, Dashboard & Deployment: FastAPI, experiment dashboard, public website, file storage/versioning, testing, deployment and workload connector.

Key risks: Incomplete carbon data, scheduler complexity, insufficient calibration data, dashboard distraction, connector failure, reproducibility issues and deployment problems.

Mitigation: Freeze the data source; start with simple baselines; use repeated controlled traces; build the simulator first; start with a simple connector; version everything; deploy a minimal API early.

Conclusion & Next Steps

Summary: CarbonRoute is a software-only framework for uncertainty-aware carbon-aware batch scheduling under imperfect forecasts.

Core contribution: Combine scheduling, deadline-risk estimation, calibration and reproducible benchmark evaluation.

Final deliverables: Scheduling API, experiment dashboard, reproducible benchmark, public project website and real workload connector.

Immediate next steps: Freeze data-source and experiment specifications; implement the simulator and required baselines; define the uncertainty/risk model; establish the reproducibility workflow.

Expected outcome: Demonstrate which schedule is selected, why it is selected, which constraints affect it, how uncertain the decision is and whether predicted risk matches observed outcomes.

References

1. **CarbonAware Scheduler Documentation:** <https://docs.carbonaware.dev/scheduler/>
2. **CarbonAware Python Scheduler Documentation:** <https://docs.carbonaware.dev/scheduler/python/>
3. **Electricity Maps API Documentation:** <https://app.electricitymaps.com/api/docs/reference>
4. **SimPy Documentation:** <https://simpy.readthedocs.io/>
5. **NIST:** Digital Twins and Digital-Twin Systems.
6. **Research literature:** Carbon-aware scheduling, forecast uncertainty, constrained optimization and deadline-risk calibration.